

09 CSharp Coding Standards

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

09 CSharp Coding Standards

6

title: "Cartridge & Cloud — C# Coding Standards" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-01" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" csharp_language_version: "9.0" target_framework: "netstandard2.1" application_version_reference: "0.0.26"	6
Cartridge & Cloud — C# Coding Standards	6
0. Propósito	6
0.1. Jerarquía de autoridad	7
0.2. Palabras normativas	7
0.3. Estado técnico auditado	7
1. Principios generales	8
1.1. Legibilidad antes que concisión	8
1.2. Dependencias explícitas	8
1.3. Una única fuente de verdad	9
1.4. Validar antes de mutar	9
1.5. Determinismo observable	9
1.6. Fallar de forma segura	9
1.7. Testabilidad como requisito de diseño	10
1.8. Cambios pequeños y trazables	10
2. Compatibilidad de lenguaje y runtime	10
2.1. Baseline	10
2.2. Características permitidas	10
2.3. Características restringidas	11
2.4. Nullable reference types	11
3. Namespaces, carpetas y assemblies	11
3.1. Namespace raíz	11
3.2. Correspondencia con carpetas	12
3.3. Assemblies de producción	12
3.4. Reglas de dependencia	12
3.5. Nuevos assemblies	13
3.6. autoReferenced	13
4. Organización de archivos y tipos	13
4.1. Un tipo principal por archivo	13
4.2. Tamaño de archivo	14
4.3. Orden interno de un tipo	14
4.4. #region	14
4.5. Usings	14
5. Convenciones de nombres	15
5.1. Tabla general	15
5.2. Nombres de tipos	15
5.3. IDs	16
5.4. Abreviaturas	16
5.5. Nombres que expresan unidades	16
6. Formato y estilo	17
6.1. Sangría y llaves	17
6.2. Longitud de línea	17
6.3. var	17
6.4. Expresiones compactas	17
6.5. Condiciones	17
6.6. Operadores y comparaciones	18
6.7. Magic numbers	18
7. Visibilidad y encapsulación	18
7.1. Mínima visibilidad	18

7.2. Campos públicos	18
7.3. Propiedades	18
7.4. Colecciones	19
7.5. Herencia	19
7.6. Clases selladas	19
8. Entidades, value objects y resultados	19
8.1. Entidades	19
8.2. Value objects	20
8.3. Dinero	20
8.4. Quantities	20
8.5. Result objects	20
8.6. Enums	21
8.7. Estados incompatibles	21
9. Null, argumentos y errores	21
9.1. Validación de argumentos	21
9.2. Excepción frente a resultado	21
9.3. Try methods	22
9.4. Captura de excepciones	22
9.5. Mensajes	22
10. Mutación, atomicidad e idempotencia	22
10.1. Atomicidad	22
10.2. Rollback	23
10.3. Idempotencia	23
10.4. Preflight y commit	23
10.5. Postcondiciones	23
11. Colecciones, LINQ y asignaciones	24
11.1. Colección adecuada	24
11.2. LINQ	24
11.3. Enumeración múltiple	24
11.4. Snapshots	24
11.5. Cachés	24
12. Eventos y suscripciones	25
12.1. Uso apropiado	25
12.2. Nombres	25
12.3. Suscripción	25
12.4. Publicación	25
12.5. Event buses	25
13. Async, coroutines y cancelación	26
13.1. Estado actual	26
13.2. Coroutines	26
13.3. async / Task	26
13.4. Cancelación	26
14. Reglas de Unity	26
14.1. MonoBehaviour fino	26
14.2. Lifecycle	27
14.3. Campos serializados	28
14.4. Métodos Configure	28
14.5. Búsquedas de escena	28
14.6. Nombres de GameObject	28
14.7. GetComponent	29
14.8. Destrucción e instanciación	29
14.9. Tiempo	29
15. ScriptableObject y autoría de contenido	29
15.1. Uso	29
15.2. Progreso mutable	29
15.3. IDs en assets	30
15.4. OnValidate	30
15.5. AssetDatabase	30
16. Escenas, prefabs y composición	30

16.1. Composition root	30
16.2. ApplicationRoot	31
16.3. StoreInitialSceneContext	31
16.4. Prefabs	31
16.5. Instaladores y builders	31
17. Input y UI	32
17.1. Propiedad exclusiva	32
17.2. Puntero sobre UI	32
17.3. UI fina	32
17.4. Listas y dropdowns	32
17.5. Foco y cierre	32
18. Persistencia y serialización	33
18.1. Separación	33
18.2. DTOs y records de save	33
18.3. Schema	33
18.4. Escritura atómica	33
18.5. Recuperación	34
18.6. Restore en dos fases	34
18.7. Serialización de enums	34
18.8. Fechas	34
19. Logging y diagnóstico	34
19.1. Capas	34
19.2. Niveles	34
19.3. Contexto	35
19.4. Spam	35
19.5. Datos sensibles	35
20. Rendimiento y memoria	35
20.1. Medir antes de optimizar	35
20.2. Rutas críticas	36
20.3. Allocations	36
20.4. Pooling	36
20.5. Materiales	36
20.6. String building	36
20.7. Profiler markers	36
21. Código de Editor	37
21.1. Aislamiento	37
21.2. Idempotencia	37
21.3. Undo	37
21.4. Validación previa	37
21.5. Reporte	37
21.6. Herramientas grandes	37
22. Preprocesador y código condicionado	38
22.1. #if UNITY_EDITOR	38
22.2. UNITY_INCLUDE_TESTS	38
22.3. Símbolos propios	38
23. Comentarios y documentación XML	38
23.1. Comentarios	38
23.2. XML documentation	38
23.3. TODO	39
23.4. Comentarios de deuda temporal	39
24. Tests C#	39
24.1. Nombre	39
24.2. Arrange / Act / Assert	39
24.3. Un comportamiento por test	39
24.4. Casos mínimos	40
24.5. EditMode	40
24.6. PlayMode	40
24.7. Limpieza	40
24.8. Tiempo y aleatoriedad	40

24.9. Assertions	40
24.10. Tests de regresión	41
24.11. No adaptar expectativas al bug	41
25. Diseño de APIs y constructores	41
25.1. Contratos pequeños	41
25.2. Constructores	41
25.3. Factories	42
25.4. Parámetros	42
25.5. Parámetros booleanos	42
25.6. Valores de retorno	42
25.7. Métodos con efectos	43
25.8. API síncrona frente a diferida	43
26. Igualdad, hashing y orden	43
26.1. Igualdad de value objects	43
26.2. Hash codes	43
26.3. Comparación	43
26.4. Floats y vectores	43
26.5. Orden determinista	44
27. Strings, localización y formato	44
27.1. Strings de dominio	44
27.2. Comparación	44
27.3. Normalización	44
27.4. Localización	44
27.5. Interpolación	45
27.6. Formato de dinero y números	45
27.7. Rutas	45
28. Threading, concurrencia y acceso a Unity	45
28.1. Hilo principal	45
28.2. Estado compartido	45
28.3. Save en background	46
28.4. Reentrancia	46
28.5. Transiciones de escena	46
29. Seguridad de datos y robustez de archivos	46
29.1. Entrada no confiable	46
29.2. Límites	46
29.3. Archivos temporales	47
29.4. Backups	47
29.5. Errores de permisos y disco	47
29.6. Datos de usuario	47
30. Código generado y dependencias externas	47
30.1. Código generado	47
30.2. Paquetes externos	47
30.3. Wrappers	47
30.4. Copiar código externo	48
31. Deprecación y migración de APIs	48
31.1. Obsolete	48
31.2. Migración gradual	48
31.3. Compatibilidad de save	48
31.4. Bridges temporales	48
32. Documentación de decisiones técnicas	49
32.1. Cuando crear ADR	49
32.2. Contenido mínimo	49
32.3. Código y ADR	49
33. Patterns aprobados	49
33.1. Service de aplicación	49
33.2. Repository adapter	50
33.3. View/controller	50
33.4. Proyección de UI	50
33.5. Scene context	50

34. Anti-patrones prohibidos	50
34.1. God Manager	51
34.2. Singleton indiscriminado	51
34.3. Service locator	51
34.4. Lógica en UI	51
34.5. Saves con referencias Unity	51
34.6. Strings como estado	51
34.7. Flags incompatibles	51
34.8. Búsquedas globales como DI	51
34.9. Excepciones ignoradas	51
34.10. Mutación pública de colecciones	51
34.11. PlayerPrefs para estado de partida	52
34.12. Código muerto protegido por comentario	52
34.13. Refactor oportunista de gran escala	52
35. Warnings, analyzers y calidad estática	52
35.1. Warnings	52
35.2. Estado de automatización	52
35.3. Regla de adopción	52
36. Refactorización y deuda	52
36.1. Refactor seguro	52
36.2. Refactor por seams	53
36.3. Deuda observada	53
36.4. No refactorizar por métrica aislada	53
37. Revisión de código	53
37.1. Checklist funcional	53
37.2. Checklist arquitectónico	53
37.3. Checklist Unity	54
37.4. Checklist datos	54
37.5. Checklist QA	54
37.6. Checklist rendimiento	54
38. Definition of Done de una contribución C#	54
39. Política de excepciones	55
40. Ejemplos de decisiones comunes	55
40.1. ¿Result o excepción?	55
40.2. ¿Domain o Application?	55
40.3. ¿ScriptableObject o save?	56
40.4. ¿Evento o llamada explícita?	56
41. Plantilla de tipo de dominio	56
42. Plantilla de componente Unity	57
43. Plantilla de test	58
44. Plan recomendado para automatizar el estándar	58
44.1. Fase 1 — Formato base	58
44.2. Fase 2 — Warnings nuevos	58
44.3. Fase 3 — Analyzers	59
44.4. Fase 4 — CI	59
45. Criterios de aceptación del estándar	59
Anexo A. Assemblies observados	60
Anexo B. Archivos grandes observados	60
Anexo D. Regla de mantenimiento	61
Anexo E. Matriz de aplicación práctica	61
E.1. Cambio de dominio	61
E.2. Cambio de Application	61
E.3. Cambio de Presentation o UI	62
E.4. Cambio de escena o prefab	62
E.5. Cambio de persistencia	62
E.6. Cambio de herramienta de Editor	63
E.7. Corrección urgente	63
E.8. Revisión de deuda	63
E.9. Evidencia de cumplimiento	63

09 CSharp Coding Standards

Metadatos normativos v2.0

document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — C# Coding Standards" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-01" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" csharp_language_version: "9.0" target_framework: "netstandard2.1" application_version_reference: "0.0.26"

Cartridge & Cloud — C# Coding Standards

0. Propósito

Este documento establece la norma vigente para escribir, revisar, probar, integrar y mantener código C# en *Cartridge & Cloud*.

Su objetivo no es imponer preferencias cosméticas aisladas. Su función es proteger:

- la separación arquitectónica;
- las invariantes de dominio;
- la integridad de inventario, dinero, reservas y persistencia;
- la testabilidad;
- el determinismo;
- la compatibilidad con Unity;
- la legibilidad para mantenimiento individual a largo plazo;
- la capacidad de producir builds reproducibles;
- la trazabilidad entre diseño, implementación y QA.

La norma se aplica a:

- código de producción;
- herramientas de Editor;
- tests EditMode y PlayMode;
- scripts de migración;
- código de autoría de contenido;
- DTOs de persistencia;
- bridges temporales;
- prototipos que vayan a integrarse en el repositorio principal.

Un prototipo descartable fuera del repositorio puede relajar estas reglas. En el momento en que su código se integra, pasa a estar sujeto a este documento.

0.1. Jerarquía de autoridad

La consolidación aplica el siguiente orden:

1. 00_Enfoque_y_Alcance.md;
2. 01_Game_Design_Document.md;
3. 02_Vertical_Slice_Specification.md;
4. 03_Technical_Design_Document.md;
5. 04_Modelo_de_Datos.md;
6. 05_UX_Flow.md;
7. 06_Production_Roadmap_y_Sprint_Plan.md;
8. 07_QA_Testing_Plan.md;
9. 08_QA_Testing_Matrix.xlsx;
10. C# Coding Standards v0.5;
11. ADR vigentes;
12. código y assemblies reales de la baseline;
13. C# Coding Standards v0.4;
14. C# Coding Standards v0.3 de baseline v0.4;
15. copia v0.3 de baseline v0.3, solo para trazabilidad.

Cuando exista una contradicción:

- prevalece una invariante funcional aprobada sobre una preferencia de estilo;
- prevalece una decisión arquitectónica vigente sobre una práctica histórica;
- el código existente no convierte automáticamente una deuda en norma;
- una norma nueva no obliga a refactorizar todo el proyecto de una vez;
- una API pública persistente no se renombra sin migración;
- una corrección de formato no debe mezclarse con una corrección funcional de alto riesgo;
- los ejemplos de este documento son normativos en intención, no plantillas que deban copiarse literalmente.

0.2. Palabras normativas

- **DEBE / NO DEBE:** requisito obligatorio.
- **DEBERÍA / NO DEBERÍA:** norma recomendada; desviarse exige motivo.
- **PUEDE:** opción permitida.
- **DEUDA:** desviación conocida que debe registrarse.
- **EXCEPCIÓN:** desviación aprobada para un caso concreto.

0.3. Estado técnico auditado

La auditoría de los ZIP del proyecto identifica:

Métrica	Valor observado
Archivos .cs	457
Líneas C# aproximadas	81,780
Assemblies .asmdef	12
Tipos derivados de MonoBehaviour	56
Tipos derivados de ScriptableObject	25
Métodos Update	9
Métodos LateUpdate	2
Campos [SerializeField]	171
readonly struct	53
Interfaces declaradas	19
Enums declarados	68
Declaraciones de eventos	11

Métrica	Valor observado
Archivos que usan LINQ	10
Archivos que usan IEnumerator	14

La configuración generada por Unity muestra:

- C# 9.0;
- netstandard2.1 para assemblies de producción;
- Unity 6.3 LTS 6000.3.18f1;
- UNITY_INCLUDE_TESTS en contextos de test;
- ausencia de .editorconfig, .globalconfig, ruleset o analyzer configurado en los paquetes revisados.

Esta ausencia es **deuda de automatización de estilo**. Hasta que se incorpore una configuración aprobada, este documento es la fuente normativa y la revisión se realiza mediante IDE, revisión manual, compilación, tests y gates.

1. Principios generales

1.1. Legibilidad antes que concisión

El código debe optimizarse para que una persona pueda entender:

- qué estado se lee;
- qué validaciones se realizan;
- qué mutaciones ocurren;
- qué puede fallar;
- qué resultado se devuelve;
- qué capa es propietaria de la decisión.

Se debe evitar comprimir varias decisiones de dominio en una única expresión.

Correcto:

```
InventoryTransferResult result =
    _transferService.Transfer(
        source,
        destination,
        productId,
        quantity);

if (!result.Succeeded)
{
    return RestockResult.Failure(
        MapFailure(result.FailureReason));
}
```

No recomendado:

```
return _transferService.Transfer(a, b, id, q).Succeeded
    ? RestockResult.Success()
    : RestockResult.Failure(RestockFailureReason.Unknown);
```

La segunda forma pierde contexto y puede ocultar la traducción correcta del fallo.

1.2. Dependencias explícitas

Una dependencia debe entrar mediante:

- constructor en tipos C# puros;

- método de composición claramente limitado;
- referencia serializada en componentes de escena;
- parámetro de caso de uso;
- interfaz implementada por infraestructura.

No debe descubrirse mediante búsqueda global cada vez que se necesita.

1.3. Una única fuente de verdad

Un dato mutable debe tener un propietario autoritativo.

La UI, una vista 3D, un MonoBehaviour o un ScriptableObject no deben mantener copias independientes de:

- dinero;
- inventario;
- reservas;
- estado de pedido;
- estado de jornada;
- asignación de display;
- posición persistente;
- progreso de tutorial;
- slot activo.

Las capas de presentación pueden mantener proyecciones o cachés, pero deben poder reconstruirse sin convertirse en autoridad.

1.4. Validar antes de mutar

Toda operación compuesta debe seguir:

```
validar argumentos  
→ resolver dependencias  
→ comprobar precondiciones  
→ construir plan o preflight  
→ aplicar mutación  
→ verificar postcondiciones críticas  
→ devolver resultado
```

No debe:

```
mutar A  
→ descubrir que B falla  
→ intentar deshacer parcialmente
```

1.5. Determinismo observable

Con las mismas entradas y el mismo estado inicial, una operación de dominio debe producir el mismo resultado.

Las fuentes de tiempo, aleatoriedad, disco, escena e input deben abstraerse o quedar fuera del dominio.

1.6. Fallar de forma segura

Un fallo esperado de gameplay debe:

- no lanzar una excepción como flujo normal;
- no dejar estado parcial;
- devolver una razón tipada;
- permitir feedback localizado;
- conservar datos válidos.

Un estado imposible o configuración inválida debe fallar temprano y con contexto suficiente.

1.7. Testabilidad como requisito de diseño

Un tipo difícil de probar suele señalar:

- demasiadas responsabilidades;
- dependencias ocultas;
- uso excesivo de estado global;
- acoplamiento a Unity;
- mutación no encapsulada;
- reloj o RNG no controlables;
- métodos demasiado grandes.

No se debe crear una API de test que debilite invariantes de producción.

1.8. Cambios pequeños y trazables

Una contribución debe intentar limitarse a una intención principal:

- nueva capacidad;
- corrección;
- migración;
- refactor;
- test;
- herramienta.

No se debe mezclar un formateo masivo con un cambio funcional si dificulta revisar la lógica.

2. Compatibilidad de lenguaje y runtime

2.1. Baseline

La baseline vigente utiliza:

Unity: 6000.3.18f1

C#: 9.0

Target de producción: netstandard2.1

Plataforma inicial: Windows x64

El código debe compilar bajo la versión de C# configurada por Unity, no bajo la versión más reciente disponible en un SDK externo.

2.2. Características permitidas

Se permiten, cuando mejoran claridad y son compatibles con Unity:

- pattern matching;
- switch expressions simples;
- expression-bodied members pequeños;
- readonly struct;
- tuples locales con nombres claros;
- inicializadores de colección;
- nameof;
- using declaration cuando Unity y el contexto lo soporten;
- tipos genéricos;
- Span<T> solo si se valida compatibilidad y necesidad;
- checked en aritmética crítica;

- `Array.Empty<T>()`;
- null conditional para observación no autoritativa.

2.3. Características restringidas

dynamic No debe utilizarse en código de producción salvo integración externa inevitable y encapsulada.

Reflexión Puede utilizarse en:

- Editor;
- serialización controlada;
- tests;
- integración técnica justificada.

No debe ser el mecanismo ordinario para resolver servicios, contenido o comportamiento.

unsafe Requiere ADR o justificación equivalente, profiling y pruebas específicas.

Records de C# Aunque C# 9 los soporta, no deben introducirse automáticamente en DTOs serializados por Unity o JSON sin verificar:

- constructor usado por el serializer;
- mutabilidad necesaria;
- compatibilidad AOT;
- semántica de igualdad;
- tamaño del cambio de schema.

Los nombres que terminan en `Record` existentes no implican necesariamente el uso de la palabra clave `record`.

2.4. Nullable reference types

La baseline no declara una política global de nullable mediante `.editorconfig` o proyecto manual. Hasta que se apruebe:

- se deben validar referencias en límites públicos;
- una referencia que puede ser nula debe documentarse o expresarse mediante un resultado;
- no se debe usar `null` como estado ambiguo cuando existe un `enum` o tipo de opción más claro;
- no debe activarse nullable de forma aislada en archivos sin plan de adopción;
- una futura adopción debe realizarse por `assembly` o fase y acompañarse de tests.

3. Namespaces, carpetas y assemblies

3.1. Namespace raíz

Todo código propio utiliza:

`VRMGames.CartridgeAndCloud`

Formato:

`VRMGames.CartridgeAndCloud.<Layer>.<Feature>`

Ejemplos:

namespace VRMGames.CartridgeAndCloud.Domain.Inventory
namespace VRMGames.CartridgeAndCloud.Application.Checkout
namespace VRMGames.CartridgeAndCloud.Infrastructure.Persistence
namespace VRMGames.CartridgeAndCloud.Presentation.Placement
namespace VRMGames.CartridgeAndCloud.Tests.EditMode.Economy

3.2. Correspondencia con carpetas

La carpeta y el namespace deberían expresar la misma capa y feature.

Scripts/Domain/Inventory/Quantity.cs
 → VRMGames.CartridgeAndCloud.Domain.Inventory

Se admite una desviación temporal cuando existe una migración registrada, pero no debe crearse una nueva desviación sin motivo.

3.3. Assemblies de producción

Assembly	Responsabilidad	Dependencias permitidas
Domain	reglas, entidades, value objects, invariantes	BCL compatible
Application	casos de uso y coordinación	Domain
Infrastructure	archivos, codecs, adapters y repositorios	Domain, Application
Presentation	vistas, controladores y componentes Unity	Domain, Application, Unity
Infrastructure.InputSystem	adapter del Input System	Application, Infrastructure
Runtime.VerticalSlicePhase1	composición runtime transitoria	capas necesarias de producción
Editor	authoring e instaladores	capas necesarias, solo Editor

La referencia actual de Infrastructure.InputSystem hacia Presentation está registrada como una dependencia a revisar. No debe utilizarse como precedente para nuevas dependencias ascendentes.

3.4. Reglas de dependencia

Domain No debe referenciar:

- UnityEngine;
- UnityEditor;
- MonoBehaviour;
- ScriptableObject;
- UI;
- disco;
- PlayerPrefs;
- escenas;
- Input System;
- servicios concretos de infraestructura.

Application No debe contener MonoBehaviour ni depender de una escena.

Puede depender de interfaces que representen:

- reloj;
- repositorio;
- navegación abstracta;
- captura y restauración;
- feedback abstracto.

Infrastructure Implementa contratos. No decide reglas de gameplay.

Presentation Traduce input y estado visual. No debe ser autoridad de inventario, dinero, reservas o jornada.

Runtime composition Puede conectar capas, pero no debe convertirse en un “God Manager” que acumule reglas.

Editor Puede conocer tipos de producción para autorar y validar, pero su código no debe compilar en Player.

3.5. Nuevos assemblies

Un nuevo `.asmdef` requiere una razón:

- frontera de dependencia;
- aislamiento de Editor;
- paquete externo;
- velocidad de compilación material;
- test assembly;
- feature que debe poder retirarse.

No se crea un assembly por cada carpeta pequeña.

3.6. autoReferenced

Los assemblies propios deberían usar `autoReferenced: false` cuando la composición explícita sea viable. Toda referencia debe ser intencional.

4. Organización de archivos y tipos

4.1. Un tipo principal por archivo

Debe existir un tipo público principal cuyo nombre coincida con el archivo.

Se permiten tipos públicos adicionales cuando son inseparables del contrato principal, por ejemplo:

- enum de estado;
- resultado y razón de fallo;
- pequeño value object;
- interfaz estrechamente asociada;
- entrada serializada anidada.

Cuando un archivo supera una responsabilidad o dificulta navegación, debe dividirse.

Ejemplo aceptable:

```
CheckoutService.cs
├─ CheckoutFailureReason
├─ CheckoutResult
└─ CheckoutService
```

Ejemplo que debería dividirse:

```
StoreEverything.cs
├─ inventario
├─ clientes
└─ economía
```

```
|— UI
|— guardado
|— audio
```

4.2. Tamaño de archivo

No existe un límite mecánico universal, pero un archivo de producción por encima de unas 500 líneas exige revisar:

- responsabilidades;
- regiones conceptuales;
- capacidad de test;
- dependencias;
- composición;
- duplicación.

Un archivo por encima de 1.000 líneas se considera señal de deuda salvo que sea:

- código generado;
- herramienta de migración excepcional;
- fixture de test amplio justificado;
- catálogo declarativo inevitable.

La auditoría observa varios archivos de más de 1.000 líneas, especialmente herramientas de Editor, UI y builders de la fase representativa. Deben tratarse como candidatos de división, no como plantilla.

4.3. Orden interno de un tipo

Orden recomendado:

1. constantes;
2. campos estáticos;
3. campos serializados;
4. campos privados;
5. constructores;
6. propiedades;
7. eventos;
8. lifecycle de Unity;
9. API pública;
10. métodos internos;
11. helpers privados;
12. tipos anidados.

No se debe sacrificar cohesión para cumplir el orden de forma rígida.

4.4. #region

No debe utilizarse para ocultar un tipo excesivamente grande.

Puede utilizarse con moderación en:

- código generado;
- herramientas de Editor largas en proceso de división;
- grandes grupos de tests cuando mejora navegación.

4.5. Usings

Orden:

1. System;
2. librerías externas;
3. UnityEngine / UnityEditor;
4. namespaces propios.

No deben permanecer usings no utilizados.

5. Convenciones de nombres

5.1. Tabla general

Elemento	Convención	Ejemplo
namespace	PascalCase por segmento	Domain.Inventory
clase	PascalCase, sustantivo	InventoryContainer
interfaz	I + PascalCase	ISaveGameRepository
struct	PascalCase	Money
enum	PascalCase singular	StoreDayState
miembro de enum	PascalCase	BeforeOpen
método	PascalCase, verbo	TryReserve
propiedad	PascalCase, sustantivo/estado	AvailableCapacity
campo privado	_camelCase	_quantities
parámetro	camelCase	productId
variable local	camelCase	previousQuantity
constante	PascalCase	CurrentSchemaVersion
evento	sustantivo + pasado o acción clara	StateChanged
handler	On + evento	OnStateChanged
bool	pregunta positiva	IsOpen, HasBackup
método booleano	verbo/pregunta	CanPlace, ContainsProduct
test	Member_Scenario_Expected	Transfer_InsufficientStock_FailsAtomically

5.2. Nombres de tipos

Un tipo debe comunicar su rol:

- Service: coordina un caso de uso;
- Repository: persiste o recupera;
- Registry: posee una colección identificada;
- Catalog: definición estática indexada;
- Policy: reglas configurables sin estado de proceso;
- Snapshot: representación inmutable o autocontenida;
- State: estado mutable o máquina de estados;
- Result: resultado de una operación;
- FailureReason: razón tipada;
- Definition: contenido estático;
- Instance: entidad runtime;
- Controller: adapta input o lifecycle;
- View: representa estado;
- CompositionRoot: conecta dependencias;
- Builder: construye un objeto o snapshot, no gobierna toda la aplicación.

No se deben usar nombres genéricos como:

- Manager;

- Helper;
- Utils;
- Data;
- Handler;
- Thing;
- System;

salvo que el contexto delimite con precisión su responsabilidad.

5.3. IDs

Los tipos de ID se nombran:

```
ProductDefinitionId  
CustomerInstanceId  
CheckoutTransactionId
```

Los IDs de contenido nuevos deben utilizar **kebab-case estable** con prefijo o dominio cuando sea necesario:

```
product-retro-console-a  
furniture-wall-shelf-tier-e  
customer-profile-collector
```

Reglas:

- no usar el nombre de GameObject como ID;
- no usar índice de lista como identidad persistente;
- no regenerar IDs al cargar;
- no renombrar IDs publicados sin migración;
- comparar IDs de string con `StringComparison.Ordinal`;
- normalizar en el constructor del value object, no en cada consumidor.

5.4. Abreviaturas

Se permiten abreviaturas ampliamente comprendidas y estables:

- Id;
- UI;
- UX;
- DTO en documentación, aunque los tipos deberían tener nombre de dominio;
- UTC;
- JSON;
- VFX.

Evitar abreviaturas internas opacas como mgr, svc, cfg, tmp, obj en API pública.

5.5. Nombres que expresan unidades

Cuando una magnitud no sea obvia, el nombre incluye unidad:

```
int durationSeconds  
float cellSizeMeters  
long minorUnits  
int timeoutMilliseconds
```

No mezclar unidades sin conversión explícita.

6. Formato y estilo

6.1. Sangría y llaves

- cuatro espacios;
- no tabs;
- llaves en línea separada;
- una instrucción por línea;
- newline final;
- UTF-8;
- evitar whitespace final.

```
if (!result.Succeeded)
{
    return result;
}
```

6.2. Longitud de línea

Objetivo recomendado: 100-120 caracteres.

Las expresiones de dominio deben dividirse por significado, no de forma aleatoria.

```
CheckoutResult result =
    _checkoutService.TryCheckout(
        stationId,
        customerId,
        expectedTransactionId);
```

6.3. var

Se permite cuando el tipo es evidente y no elimina lenguaje de dominio:

```
var snapshot = builder.Build();
```

Se prefiere tipo explícito cuando:

- el método devuelve un tipo no obvio;
- el nombre local es genérico;
- el tipo es parte de la explicación;
- se comparan tipos similares;
- se revisa código crítico.

```
InventoryTransferResult result =
    service.Transfer(source, destination, productId, quantity);
```

No se debe prohibir var por dogma ni utilizarlo por defecto.

6.4. Expresiones compactas

Los expression-bodied members son apropiados para propiedades o métodos triviales:

```
public bool IsEmpty => Count == 0;
```

No se usan para operaciones con validación, efectos o varios pasos.

6.5. Condiciones

Preferir retornos tempranos para reducir anidación.

```
if (request == null)
{
    throw new ArgumentNullException(nameof(request));
}
```

```
if (!request.IsValid)
{
    return OrderCreationResult.Failure(
        OrderCreationFailureReason.InvalidRequest);
}
```

No combinar demasiadas razones distintas en una única condición si deben producir feedback diferente.

6.6. Operadores y comparaciones

- usar `==` `null` o `is null` de forma consistente por contexto;
- usar `StringComparison.Ordinal` para IDs;
- usar `OrdinalIgnoreCase` solo para entrada humana apropiada;
- no comparar floats directamente cuando existe tolerancia;
- usar `checked` para dinero, cantidades acumuladas y cálculos que no admiten overflow.

6.7. Magic numbers

Todo número que exprese regla, unidad, límite o balance debe:

- tener nombre;
- vivir en una `policy`/configuración cuando deba ajustarse;
- documentar su unidad;
- tener tests de límite.

Valores triviales como 0, 1 o índices locales pueden permanecer literales si su significado es evidente.

7. Visibilidad y encapsulación

7.1. Mínima visibilidad

Utilizar la visibilidad más restrictiva que permita la responsabilidad:

- `private` por defecto;
- `internal` para colaboración dentro de `assembly` cuando proceda;
- `public` solo para contrato necesario;
- `protected` con moderación.

7.2. Campos públicos

No se permiten campos públicos mutables en código de producción.

Correcto:

```
[SerializeField]
private Transform _checkoutAnchor;

public Transform CheckoutAnchor => _checkoutAnchor;
```

Aun así, exponer un `Transform` públicamente debe estar justificado por la composición.

7.3. Propiedades

Una propiedad no debe:

- ejecutar I/O;
- mutar estado ocultamente;

- instanciar objetos pesados repetidamente;
- lanzar excepciones inesperadas por simple lectura.

Las propiedades calculadas pequeñas son apropiadas.

7.4. Colecciones

No exponer una colección mutable interna.

Opciones:

- `IReadOnlyList<T>`;
- snapshot mediante array;
- enumeración controlada;
- métodos de consulta;
- `ReadOnlyCollection<T>` cuando aporte valor.

Advertencia: devolver `IReadOnlyList<T>` sobre una `List<T>` interna no impide que otro código con referencia original la modifique. La propiedad real debe seguir encapsulada.

7.5. Herencia

Preferir composición.

Usar herencia cuando:

- existe una relación estable “es un”;
- la clase base define un contrato coherente;
- no se requiere inspeccionar tipo concreto para funcionar;
- la sustitución es válida.

No crear jerarquías profundas de `MonoBehaviour` para compartir pequeñas utilidades.

7.6. Clases selladas

Las clases sin diseño explícito de herencia deberían ser sealed, especialmente:

- services;
- repositories concretos;
- value containers;
- componentes con lifecycle específico.

8. Entidades, value objects y resultados

8.1. Entidades

Una entidad:

- tiene ID estable;
- protege invariantes;
- es propietaria de su transición;
- no expone setters arbitrarios;
- no depende de representación Unity.

```
public sealed class CustomerInstance
{
    public CustomerInstanceId Id { get; }
    public CustomerState State { get; private set; }
```

```
public CustomerTransitionResult TryEnterQueue(...)
{
    // Validar y mutar atómicamente.
}
}
```

8.2. Value objects

Un value object debe:

- validar en construcción;
- ser inmutable;
- implementar igualdad por valor;
- representar una unidad conceptual;
- impedir combinaciones inválidas.

Los readonly struct existentes para Money, Quantity e IDs son patrones válidos.

Precauciones:

- un struct no inicializado existe como default;
- el código debe detectar un Value vacío cuando default no sea válido;
- no crear structs grandes;
- no usar structs mutables;
- no almacenar referencias mutables dentro de un value object sin necesidad.

8.3. Dinero

El dinero usa unidades menores enteras:

```
public readonly struct Money
{
    public long MinorUnits { get; }
    public CurrencyCode Currency { get; }
}
```

Reglas:

- nunca float o double para contabilidad;
- misma moneda antes de sumar o comparar;
- overflow comprobado;
- redondeo explícito en conversiones;
- formato localizado solo en Presentation;
- persistencia en unidad menor y código de moneda.

8.4. Quantities

Las cantidades de inventario deben ser enteras y no negativas.

No usar un int desnudo si el value object evita estados inválidos o confusión entre:

- unidades;
- cajas;
- capacidad;
- slots;
- cantidad reservada.

8.5. Result objects

Una operación esperablemente rechazable devuelve un resultado tipado.

```
public readonly struct InventoryTransferResult
{
    public bool Succeeded { get; }
    public InventoryTransferFailureReason FailureReason { get; }
}
```

Reglas:

- Succeeded y FailureReason deben ser coherentes;
- el éxito no utiliza una razón de fallo real;
- las razones no se expresan con strings;
- un resultado puede transportar IDs o valores necesarios para continuar;
- Presentation traduce la razón a feedback.

8.6. Enums

Un enum debe:

- incluir Unspecified = 0 cuando default no sea válido;
- usar nombres singulares;
- validarse al recibir datos externos;
- no persistirse como índice implícito si el orden puede cambiar;
- no utilizar flags salvo combinación semánticamente válida.

8.7. Estados incompatibles

Evitar múltiples bools que permiten combinaciones imposibles:

```
bool isOpen;
bool isClosing;
bool isClosed;
```

Preferir:

```
StoreDayState state;
```

9. Null, argumentos y errores

9.1. Validación de argumentos

Un método público valida argumentos que no puede interpretar de forma segura.

- ArgumentNullException: referencia obligatoria nula;
- ArgumentException: valor formado incorrectamente;
- ArgumentOutOfRangeException: valor fuera de rango;
- InvalidOperationException: estado interno imposible para la operación.

Usar nameof.

```
if (repository == null)
{
    throw new ArgumentNullException(nameof(repository));
}
```

9.2. Excepción frente a resultado

Excepción Apropriada para:

- configuración de escena inválida;
- constructor con invariante imposible;

- dependencia obligatoria ausente;
- corrupción interna no recuperable en ese nivel;
- uso incorrecto de API por código.

Resultado Apropriado para:

- saldo insuficiente;
- inventario insuficiente;
- destino ocupado;
- cantidad inválida introducida por gameplay;
- cliente no elegible;
- estado de jornada que no permite acción;
- slot vacío o backup recuperable.

9.3. Try methods

Un método TryX:

- no lanza por un rechazo esperado;
- devuelve bool y out, o un result object;
- no deja mutación parcial;
- documenta qué fallos siguen pudiendo lanzar.

9.4. Captura de excepciones

No se permite un catch vacío.

```
catch (IOException exception)
{
    Debug.LogError(
        $"Save write failed for slot {slotId}: {exception.Message}");

    return SaveResult.Failure(
        SaveFailureReason.WriteFailed);
}
```

No debe registrarse el mismo error en cada capa. La capa que tiene contexto suficiente lo registra o lo traduce, y las capas superiores evitan duplicarlo.

9.5. Mensajes

Los mensajes de excepción deben incluir:

- operación;
- entidad o ID cuando no contiene datos sensibles;
- valor esperado;
- valor recibido cuando sea seguro;
- siguiente pista técnica.

No deben utilizarse como texto localizado para usuario.

10. Mutación, atomicidad e idempotencia

10.1. Atomicidad

Inventario, reservas, checkout, economía, placement y persistencia deben aplicar mutación atómica.

Patrón:

```
CheckoutPreflightResult preflight =  
    ValidateCheckout(request);  
  
if (!preflight.Succeeded)  
{  
    return CheckoutResult.Failure(  
        preflight.FailureReason);  
}  
  
return CommitCheckout(preflight.Plan);
```

10.2. Rollback

Se debe evitar depender de rollback para operaciones en memoria si un preflight puede garantizar la validez.

Si el rollback es inevitable:

- debe estar encapsulado;
- debe probarse con fault injection;
- debe manejar fallo del propio rollback;
- debe registrar inconsistencia crítica.

10.3. Idempotencia

Una operación idempotente debe usar una clave estable:

- transaction ID;
- posting key;
- delivery ID;
- checkpoint ID;
- generation ID.

No debe decidir idempotencia mediante “parece que ya ocurrió” examinando solo el saldo o la cantidad final.

10.4. Preflight y commit

El preflight debe ser:

- libre de efectos;
- reproducible;
- suficientemente completo;
- válido durante el commit o revalidado antes de mutar.

10.5. Postcondiciones

Después de una operación crítica, pueden verificarse en desarrollo/tests:

- conservación de unidades;
- no negatividad;
- reserva consumida una vez;
- ledger reconciliado;
- ocupación sin celdas duplicadas;
- snapshot válido.

11. Colecciones, LINQ y asignaciones

11.1. Colección adecuada

- `List<T>`: orden e iteración;
- `Dictionary<TKey, TValue>`: lookup por ID;
- `HashSet<T>`: pertenencia;
- `Queue<T>`: FIFO;
- `Stack<T>`: LIFO;
- arrays: tamaño fijo y snapshots.

La estructura debe expresar la invariante.

11.2. LINQ

LINQ se permite en:

- authoring;
- Editor;
- tests;
- construcción de snapshots poco frecuente;
- código de claridad no crítico.

Evitarlo en:

- Update;
- loops por cliente cada frame;
- rutas de spawning frecuentes;
- renderización de HUD por frame;
- operaciones que deban controlar allocations.

No encadenar consultas complejas cuando un bucle explícito es más claro o evita enumeraciones repetidas.

11.3. Enumeración múltiple

Un `IEnumerable<T>` recibido de fuera puede ser perezoso. Si se necesita recorrer varias veces:

- materializar una vez;
- o exigir una colección concreta;
- o diseñar el método para una sola pasada.

11.4. Snapshots

Cuando se expone estado:

- usar una copia estable;
- ordenar de forma determinista si la salida se persiste o compara;
- no confiar en orden de `Dictionary` como contrato de save;
- evitar reconstruir snapshots pesados cada frame.

11.5. Cachés

Una caché debe declarar:

- clave;
- propietario;
- momento de invalidación;
- impacto si queda obsoleta;

- si persiste o se reconstruye.
-

12. Eventos y suscripciones

12.1. Uso apropiado

Los eventos son apropiados para notificar que algo ya ocurrió:

- cambio de estado;
- actualización de proyección;
- cambio de selección;
- finalización de transición.

No deben coordinar operaciones críticas cuyo orden o fallo requiera control explícito.

12.2. Nombres

Preferir:

```
public event Action<StoreDayState> StateChanged;  
public event Action<Money> BalanceChanged;
```

Evitar prefijo On en la declaración del evento. OnStateChanged se reserva al método que lo publica o maneja.

12.3. Suscripción

Un MonoBehaviour que se suscribe en OnEnable debe desuscribirse en OnDisable.

```
private void OnEnable()  
{  
    _service.StateChanged += OnStateChanged;  
}  
  
private void OnDisable()  
{  
    _service.StateChanged -= OnStateChanged;  
}
```

Si la dependencia puede cambiar, debe desuscribirse de la anterior antes de sustituirla.

12.4. Publicación

Copiar el delegate localmente no es necesario en las versiones modernas para `?.Invoke`, pero el publicador debe proteger su estado antes de notificar.

12.5. Event buses

No se introduce un event bus global para evitar dependencias explícitas.

Un bus de eventos requiere:

- ámbito definido;
 - tipos de evento inmutables;
 - ownership;
 - orden no crítico;
 - estrategia de test;
 - prevención de ciclos.
-

13. Async, coroutines y cancelación

13.1. Estado actual

La baseline revisada no utiliza Task/async como patrón dominante. Las operaciones de Unity asíncronas se resuelven principalmente mediante coroutines o APIs de escena.

13.2. Coroutines

Una coroutine puede utilizarse para:

- esperar carga de escena;
- secuenciar una transición visual;
- esperar frames en PlayMode tests;
- coordinar operaciones Unity que no son lógica de dominio.

No debe:

- contener reglas autoritativas ocultas;
- sustituir una máquina de estados;
- mutar varias entidades sin coordinación;
- depender de `WaitForSeconds` para lógica económica o de jornada persistente.

13.3. async / Task

Antes de introducirlo se debe definir:

- compatibilidad de Unity;
- contexto de continuación;
- cancelación;
- excepción no observada;
- lifetime de escena;
- test;
- relación con Player shutdown.

No usar `async void` salvo event handler inevitable. Los métodos asíncronos deben terminar en `Async`.

13.4. Cancelación

Toda operación que pueda sobrevivir a una escena o pantalla debería aceptar un token o mecanismo equivalente.

Al cancelar:

- no dejar mutación parcial;
- no registrar como error un cierre esperado;
- liberar recursos;
- impedir callbacks sobre objetos destruidos.

14. Reglas de Unity

14.1. MonoBehaviour fino

Un `MonoBehaviour` debe ocuparse de:

- lifecycle;
- referencias de escena;

- traducción de input;
- conexión con servicios;
- representación visual;
- forwarding de eventos.

No debería contener:

- reglas de precio;
- conservación de inventario;
- cálculo de cierre;
- mutación autoritativa de reservas;
- serialización completa;
- validación de schema.

14.2. Lifecycle

Awake

- validar dependencias locales;
- inicializar estado interno mínimo;
- no depender de orden accidental entre objetos;
- no ejecutar casos de uso que requieren la escena completa salvo composición explícita.

OnEnable / OnDisable

- suscripciones;
- habilitar input;
- liberar suscripciones simétricamente.

Start

- trabajo que necesita otros Awake completados;
- no utilizarlo para reparar arquitectura oculta.

Update Solo para comportamiento que realmente necesita frecuencia por frame:

- lectura de input continuo;
- interpolación;
- seguimiento visual;
- temporizador visual no autoritativo.

No se debe hacer en Update:

- `FindFirstObjectByType`;
- `GameObject.Find`;
- LINQ que asigne;
- acceso a disco;
- guardar;
- reconstruir catálogos;
- resolver IDs por string;
- publicar logs repetidos.

LateUpdate Apropiado para cámara o presentación que depende de movimiento ya aplicado.

OnDestroy Solo limpieza necesaria. No confiar en él para guardar estado crítico.

14.3. Campos serializados

Patrón:

```
[SerializeField]  
private Transform _entranceAnchor;
```

Reglas:

- privados;
- nombre `_camelCase`;
- tooltip o header cuando el inspector sea complejo;
- validación en `OnValidate` o herramienta cuando aporte valor;
- no exponer setters públicos para facilitar tests;
- no usar referencias serializadas para servicios globales si el composition root puede inyectarlos.

14.4. Métodos Configure

Pueden utilizarse para:

- composición runtime;
- tests;
- installers;
- prefabs configurados programáticamente.

Deben:

- validar una sola vez o definir reconfiguración;
- no dejar estado medio configurado;
- tener nombre más específico si configura una parte;
- documentar si puede llamarse después de `Awake`.

14.5. Búsquedas de escena

`FindFirstObjectByType`, `GameObject.Find` y `Transform.Find` no son mecanismos primarios de arquitectura.

Pueden aceptarse en:

- herramientas de migración;
- reparación de assets;
- fallback técnico temporal;
- tests que buscan un objeto conocido;
- bootstrap extremadamente acotado.

Toda búsqueda de producción debe:

- ejecutarse fuera de loops calientes;
- validar exactamente un resultado;
- registrar ambigüedad;
- tener plan para referencia explícita si es estructural.

La auditoría observa búsquedas globales y por jerarquía en el código actual. Parte pertenece a herramientas e integración transitoria. No debe crecer esta dependencia.

14.6. Nombres de GameObject

No usar nombres como identidad autoritativa.

Incorrecto:

```
if (gameObject.name == "CheckoutCounter")
{
    // Decide lógica.
}
```

Correcto:

```
[SerializeField]
private CheckoutStationView _checkoutStation;
```

O referencia mediante StoreInitialSceneContext.

14.7. GetComponent

Puede utilizarse localmente cuando el contrato está en el mismo objeto o jerarquía inmediata.

- cachear el resultado;
- usar TryGetComponent para ausencia esperable;
- no repetir por frame;
- no caminar jerarquías extensas como mecanismo de DI.

14.8. Destrucción e instanciación

- Destroy en runtime;
- DestroyImmediate solo Editor/tests controlados;
- destruir objetos creados por tests;
- evitar instanciación por frame;
- utilizar pooling cuando profiling lo justifique;
- limpiar diccionarios de vistas al destruir instancias.

14.9. Tiempo

El dominio no lee `Time.time`, `Time.deltaTime` o `DateTime.UtcNow` directamente.

Presentation puede usar `deltaTime` para interpolación. Los casos de uso reciben:

- delta explícito;
- reloj `IUtcClock`;
- tiempo lógico de jornada.

15. ScriptableObject y autoría de contenido

15.1. Uso

Un `ScriptableObject` es apropiado para:

- definiciones estáticas;
- catálogos;
- configuración;
- paletas;
- referencias de prefab;
- parámetros de balance;
- assets de localización o presentación.

15.2. Progreso mutable

No almacenar como fuente de verdad:

- dinero;

- día actual;
- stock;
- reservas;
- ventas;
- progreso de slot;
- estado de cliente.

Los assets pueden ser modificados en Editor durante Play Mode. Esa mutación no debe confundirse con persistencia.

15.3. IDs en assets

Un asset authoring debe:

- tener ID estable;
- validar duplicados;
- validar referencias;
- no inferir ID del nombre de archivo en runtime;
- poder convertirse a modelo de dominio explícito.

15.4. OnValidate

Puede:

- clamp de valores de autoría;
- normalizar ID;
- validar referencias locales;
- marcar error claro.

No debe:

- recorrer todo el proyecto en cada cambio;
- guardar assets silenciosamente;
- crear objetos irreversibles;
- ejecutar reglas de gameplay complejas.

15.5. AssetDatabase

Solo Editor. Las rutas deben centralizarse y validarse.

Toda creación o modificación debe considerar:

- Undo cuando es operación manual;
- EditorUtility.SetDirty;
- AssetDatabase.SaveAssets en el punto apropiado;
- preservación de .meta;
- idempotencia;
- reejecución segura.

16. Escenas, prefabs y composición

16.1. Composition root

La composición conecta implementaciones concretas con contratos.

Debe:

- conocer dependencias;

- validar referencias;
- construir servicios en orden explícito;
- exponer solo lo necesario;
- fallar antes de devolver control al jugador.

No debe:

- contener reglas de negocio;
- usarse como service locator global;
- crecer indefinidamente.

16.2. ApplicationRoot

Es propietario del lifetime global. Debe ser único y persistente.

No se accede mediante búsquedas repetidas. Los consumidores reciben contratos o una referencia controlada durante composición.

16.3. StoreInitialSceneContext

La escena autorada debe resolver explícitamente:

- entrada;
- salida;
- recepción;
- checkout;
- roots;
- zonas;
- colliders técnicos;
- referencias visuales necesarias.

No debe deducir arquitectura mediante:

- nombres de FBX;
- bounds de renderer;
- orden de hijos;
- posición aproximada;
- nombre de GameObject.

16.4. Prefabs

Un prefab debe:

- tener responsabilidad clara;
- no duplicar managers globales;
- conservar referencias locales;
- validar componentes obligatorios;
- evitar overrides accidentales masivos;
- mantener .meta y GUID.

16.5. Instaladores y builders

Un installer de Editor debe ser idempotente:

primera ejecución → crea/configura

segunda ejecución → no duplica

estado parcial → repara o informa

estado incompatible → aborta con diagnóstico

Un builder runtime solo debe crear estado dinámico que no pertenece a la escena autorada.

17. Input y UI

17.1. Propiedad exclusiva

Solo un contexto principal procesa una entrada.

Orden:

```
modal
→ UI activa
→ overlay
→ construcción
→ interacción
→ movimiento
```

17.2. Puntero sobre UI

Los controladores de mundo deben comprobar el consumo de UI antes de raycast o confirmación.

```
if (EventSystem.current != null &&
    EventSystem.current.IsPointerOverGameObject())
{
    return;
}
```

La comprobación no reemplaza la arquitectura de contextos, pero protege contra propagación.

17.3. UI fina

La UI:

- lee una proyección;
- envía comandos/casos de uso;
- representa resultado;
- no calcula saldo autoritativo;
- no modifica colecciones del dominio;
- no persiste índices visuales;
- no concatena mensajes complejos localizados.

17.4. Listas y dropdowns

El elemento visual almacena o resuelve un ID estable. No se persiste el índice del dropdown.

17.5. Foco y cierre

Un panel debe:

- devolver contexto previo;
 - no dejar input bloqueado;
 - cancelar suscripciones;
 - ignorar callbacks después de destruirse;
 - no confirmar acciones destructivas al cerrarse.
-

18. Persistencia y serialización

18.1. Separación

El modelo persistente puede ser distinto de la entidad runtime. La raíz integrada vigente es `IntegratedGameStateSnapshot`, cuyo contrato debe permanecer explícito y versionado.

Ventajas:

- schema explícito;
- migración;
- validación;
- ausencia de referencias Unity;
- control de defaults;
- estabilidad.

18.2. DTOs y records de save

Un record de save debe contener:

- tipos serializables estables;
- IDs;
- números y enums validados;
- listas ordenadas de forma determinista cuando sea necesario;
- schema o envelope apropiado.

No debe contener:

- `GameObject`;
- `Transform`;
- `MonoBehaviour`;
- delegates;
- servicios;
- caches reconstruibles;
- referencias a assets por instancia de memoria.

18.3. Schema

Todo campo persistente nuevo exige:

1. decisión de default;
2. compatibilidad con saves anteriores;
3. cambio de schema si procede;
4. validación;
5. test de round trip;
6. test de recuperación;
7. actualización del modelo de datos.

18.4. Escritura atómica

Patrón vigente:

```
capturar  
→ validar  
→ serializar  
→ escribir temporal  
→ validar temporal  
→ rotar backup  
→ promover primario
```

No escribir directamente sobre el único archivo válido.

18.5. Recuperación

La recuperación es backup-first:

- no sobrescribir antes de decidir;
- distinguir primario inválido de backup válido;
- registrar generación;
- informar pérdida potencial;
- restaurar en dos fases.

18.6. Restore en dos fases

parsear y validar snapshot

→ construir plan de restauración

→ comprobar referencias y catálogos

→ aplicar sobre objetivo

La primera fase no muta la sesión.

18.7. Serialización de enums

Si se serializa como entero, cambios de orden rompen compatibilidad. Preferir:

- valores explícitos estables;
- strings validados cuando el coste es aceptable;
- migración.

18.8. Fechas

- UTC para timestamps técnicos;
 - formato ISO 8601 en texto;
 - no usar hora local como identidad;
 - inyectar reloj en tests.
-

19. Logging y diagnóstico

19.1. Capas

Domain no depende de Unity logs.

Los logs viven en:

- Infrastructure;
- Presentation;
- Runtime composition;
- Editor.

Application devuelve resultados; la capa con contexto decide registrar.

19.2. Niveles

Info

- transición relevante;
- build diagnostic controlado;
- recuperación completada;
- inicialización única.

Warning

- fallback;
- dato reparable;
- referencia opcional ausente;
- deuda temporal activa.

Error

- operación no recuperable en ese nivel;
- save inválido sin backup;
- referencia obligatoria ausente;
- violación de invariante;
- build flow roto.

19.3. Contexto

Un log útil incluye:

- sistema;
- operación;
- ID;
- slot/build/schema si aplica;
- razón;
- estado de fallback.

```
Debug.LogError(  
    $"[Persistence] Restore failed. " +  
    $"Slot={slotId}, Schema={schemaVersion}, " +  
    $"Reason={reason}." );
```

19.4. Spam

No registrar por frame el mismo problema.

Opciones:

- registrar una vez;
- agrupar;
- contador;
- rate limit;
- profiler marker;
- estado visible en herramienta de diagnóstico.

19.5. Datos sensibles

No registrar rutas o datos personales innecesarios. Los saves no deben volcarse completos en logs de usuario.

20. Rendimiento y memoria

20.1. Medir antes de optimizar

No se aprueba una complejidad adicional sin evidencia de profiling.

20.2. Rutas críticas

Prestar atención a:

- Update;
- navegación de clientes;
- spawning;
- consulta de displays;
- UI refrescada frecuentemente;
- visualización de stock;
- placement preview;
- serialización;
- carga de escena.

20.3. Allocations

Evitar en loops calientes:

- LINQ;
- interpolación de strings para logs;
- listas temporales;
- boxing;
- closures;
- GetComponent repetido;
- instanciación de materiales.

20.4. Pooling

Se introduce cuando:

- existe churn medido;
- el lifecycle es claro;
- reset puede garantizarse;
- no introduce estado residual.

No se usa pooling por defecto para todos los objetos.

20.5. Materiales

No acceder a `renderer.material` repetidamente si crea instancias. Utilizar:

- shared material cuando sea seguro;
- `MaterialPropertyBlock`;
- caché explícita.

20.6. String building

La UI puede formatear texto al cambiar estado. No reconstruir strings complejos cada frame.

20.7. Profiler markers

Se pueden introducir en rutas críticas de Sprint 17. Los nombres deben incluir sistema y operación.

21. Código de Editor

21.1. Aislamiento

Todo código que utiliza UnityEditor debe:

- estar en carpeta Editor o assembly solo Editor;
- no ser referenciado por Player;
- usar `#if UNITY_EDITOR` solo cuando el aislamiento por assembly no sea posible.

21.2. Idempotencia

Las herramientas de autoría deben poder reejecutarse.

No deben:

- duplicar assets;
- cambiar GUID;
- crear hijos repetidos;
- sobrescribir contenido manual sin confirmación;
- dejar una escena sucia sin indicar cambios.

21.3. Undo

Las operaciones iniciadas manualmente desde menú/inspector deberían soportar Undo cuando modifican escenas o assets.

21.4. Validación previa

Antes de modificar:

- comprobar ruta;
- comprobar tipo de asset;
- comprobar estado de escena;
- comprobar duplicados;
- crear backup o plan cuando sea destructivo.

21.5. Reporte

Una herramienta larga debe producir:

- cambios realizados;
- cambios omitidos;
- errores;
- siguiente acción;
- resultado idempotente.

21.6. Herramientas grandes

Las herramientas de más de 1.000 líneas observadas son deuda de cohesión. Deben dividirse por:

- discovery;
- validation;
- migration;
- asset creation;
- prefab wiring;
- reporting.

22. Preprocesador y código condicionado

22.1. #if UNITY_EDITOR

Preferir assemblies Editor. Usar el preprocesador para diferencias pequeñas y localizadas.

22.2. UNITY_INCLUDE_TESTS

No permitir que código de test se compile en Player por error.

Los asmdef de tests deben mantener:

- referencias correctas;
- plataformas apropiadas;
- nunit.framework.dll cuando proceda;
- aislamiento de producción.

22.3. Símbolos propios

Un símbolo propio requiere:

- dueño;
- documentación;
- configuración de build;
- test de ambas ramas;
- plan de retirada si es temporal.

No usar símbolos para esconder código muerto indefinidamente.

23. Comentarios y documentación XML

23.1. Comentarios

Comentar el **porqué**, no repetir el código.

Útil:

```
// La reserva se consume después de registrar el posting para que  
// un reintento con la misma clave permanezca idempotente.
```

Inútil:

```
// Incrementa i.  
i++;
```

23.2. XML documentation

Debe utilizarse en:

- API pública no obvia;
- interfaces compartidas;
- contratos de persistencia;
- métodos con excepciones importantes;
- herramientas expuestas a otros módulos.

No es obligatorio documentar propiedades autoexplicativas privadas.

23.3. TODO

Un TODO debe incluir referencia o condición:

```
// TODO(CC-142): retirar fallback después de cerrar StoreInitial migration.
```

No utilizar TODO como almacén indefinido de ideas. La deuda real debe existir en backlog o registro.

23.4. Comentarios de deuda temporal

Deben explicar:

- qué evita;
 - qué riesgo tiene;
 - cuándo se retira;
 - enlace o ID.
-

24. Tests C#

24.1. Nombre

Member_Scenario_ExpectedResult

Ejemplos:

Transfer_InsufficientSourceQuantity_FailsAtomically
Checkout_SameTransactionId_IsIdempotent
Load_PrimaryCorruptBackupValid_RecoversBackup

24.2. Arrange / Act / Assert

Debe existir una lectura clara aunque no se escriban comentarios.

```
[Test]
public void Transfer_InsufficientSourceQuantity_FailsAtomically()
{
    InventoryContainer source =
        CreateContainer(quantity: 2);

    InventoryContainer destination =
        CreateContainer(quantity: 0);

    InventoryTransferResult result =
        _service.Transfer(
            source,
            destination,
            ProductA,
            new Quantity(3));

    Assert.That(result.Succeeded, Is.False);
    Assert.That(source.GetQuantity(ProductA).Value, Is.EqualTo(2));
    Assert.That(destination.GetQuantity(ProductA).Value, Is.Zero);
}
```

24.3. Un comportamiento por test

Un test puede tener varias aserciones si verifican una misma postcondición.

No debe probar diez flujos independientes en un único método salvo Golden Path o integración intencional.

24.4. Casos mínimos

Para una operación de dominio:

- nominal;
- límite inferior;
- límite superior;
- argumento inválido;
- estado incompatible;
- capacidad insuficiente;
- idempotencia cuando aplica;
- conservación;
- no mutación en fallo.

24.5. EditMode

Preferido para:

- Domain;
- Application;
- codecs;
- repositorios aislables;
- reglas;
- migraciones;
- invariantes.

24.6. PlayMode

Para:

- lifecycle;
- escenas;
- input;
- componentes;
- composición;
- prefabs;
- navegación;
- integración visual funcional.

24.7. Limpieza

Todo `GameObject`, `ScriptableObject`, archivo temporal o suscripción creado por un test debe limpiarse en `finally`, `TearDown` o `UnityTearDown`.

24.8. Tiempo y aleatoriedad

- seed fija;
- reloj falso;
- no `Thread.Sleep`;
- no depender de hora local;
- esperar frames solo cuando el comportamiento Unity lo exige.

24.9. Assertions

Las aserciones deben expresar la regla. Evitar simplemente `Is.NotNull` cuando se puede verificar el comportamiento.

24.10. Tests de regresión

Todo bug S0/S1 y todo bug determinista crítico debería recibir un test antes o junto a la corrección cuando sea técnicamente razonable.

24.11. No adaptar expectativas al bug

No se cambia un resultado esperado para hacer verde un comportamiento incorrecto. Primero se revisa el contrato de diseño.

25. Diseño de APIs y constructores

25.1. Contratos pequeños

Una API pública debe exponer el mínimo necesario para ejecutar una responsabilidad. Cada método público aumenta el contrato que deberá mantenerse, probarse y migrarse.

Antes de hacer un miembro público, comprobar:

- qué consumidor lo necesita;
- si debería vivir en otra capa;
- si expone estado mutable;
- si permite saltarse invariantes;
- si será persistido o usado por tests;
- si puede ser interna.

No se añade un setter público para facilitar una prueba. Se construye el estado mediante un constructor, factory, builder de test o transición válida.

25.2. Constructores

Un constructor debe dejar el objeto en un estado válido y utilizable.

Debe:

- validar argumentos;
- copiar colecciones externas si se almacenan;
- normalizar IDs o strings una sola vez;
- evitar I/O;
- evitar acceso a escena;
- no publicar `this` antes de terminar;
- no iniciar coroutines o suscripciones.

Un constructor no debe ocultar trabajo costoso o fallos externos.

Incorrecto:

```
public StoreService()
{
    _save = File.ReadAllText("save.json");
    _root = GameObject.Find("ApplicationRoot");
}
```

Correcto:

```
public StoreService(
    IIntegratedSaveRepository repository,
    IUtcClock clock)
{
    _repository = repository ??
        throw new ArgumentNullException(nameof(repository));
}
```

```
    _clock = clock ??  
        throw new ArgumentNullException(nameof(clock));  
}
```

25.3. Factories

Una factory es apropiada cuando:

- la creación necesita varios pasos;
- hay defaults de producto;
- se asignan IDs;
- se combinan catálogos;
- la construcción puede fallar de forma esperable;
- el constructor debería permanecer simple.

La factory no debe mutar repositorios o economía salvo que su nombre y caso de uso lo declaren.

25.4. Parámetros

Evitar listas largas de parámetros del mismo tipo.

Si un método requiere más de cinco o seis argumentos, evaluar:

- request object;
- policy;
- snapshot;
- constructor de servicio;
- división de responsabilidad.

Un request object debe ser validado en el límite y no convertirse en una “bolsa” mutable de datos sin semántica.

25.5. Parámetros booleanos

Un booleano posicional suele ser ambiguo:

```
runner.Configure(settings, false);
```

Preferir, cuando el contexto no sea evidente:

```
runner.Configure(  
    settings,  
    runAutomatically: false);
```

O un enum/options type cuando existan más de dos modos.

25.6. Valores de retorno

No devolver null para indicar múltiples situaciones distintas.

Preferir:

- TryGet...;
- result object;
- colección vacía;
- enum de estado;
- snapshot explícito.

Una colección de consulta debería devolver vacía, no nula, salvo contrato externo inevitable.

25.7. Métodos con efectos

El nombre debe expresar el efecto:

- CreateOrder;
- TryReceiveDelivery;
- Save;
- Restore;
- ApplyPlacement.

No utilizar nombres como Process, Handle o Do cuando ocultan el resultado real.

25.8. API síncrona frente a diferida

Un método síncrono no debe iniciar trabajo diferido silencioso. Si la operación termina después:

- usar coroutine/task explícita;
- devolver handle o estado;
- notificar finalización;
- permitir cancelación;
- documentar lifetime.

26. Igualdad, hashing y orden

26.1. Igualdad de value objects

Un value object debe implementar igualdad coherente:

- Equals(T);
- Equals(object);
- GetHashCode;
- operadores == y != si aportan claridad.

La igualdad debe usar todos los campos que definen el valor y ninguno que sea una caché.

26.2. Hash codes

Un tipo utilizado como clave de Dictionary o miembro de HashSet debe ser inmutable mientras permanece en la colección.

No cambiar un campo que participa en GetHashCode después de insertar el objeto.

26.3. Comparación

Implementar IComparable<T> solo cuando existe un orden natural estable.

Para Money, comparar exige moneda compatible. Para IDs, normalmente no existe una prioridad de dominio; un orden ordinal puede utilizarse únicamente para serialización o presentación determinista.

26.4. Floats y vectores

No usar Vector3 == como prueba de proximidad de gameplay sin comprender su tolerancia.

Para tests o reglas:

```
Assert.That(
    Vector3.Distance(actual, expected),
    Is.LessThanOrEqualTo(tolerance));
```

El dominio debería evitar Vector3 cuando una coordenada de grid o unidad explícita representa mejor la regla.

26.5. Orden determinista

Antes de persistir o generar evidencia, ordenar colecciones por una clave estable:

```
Array.Sort(  
    records,  
    (left, right) => string.Compare(  
        left.Id,  
        right.Id,  
        StringComparison.Ordinal));
```

No confiar en el orden accidental de assets, jerarquía o diccionario.

27. Strings, localización y formato

27.1. Strings de dominio

Los strings son apropiados para:

- IDs encapsulados;
- nombres de contenido;
- claves de localización;
- mensajes técnicos;
- rutas de infraestructura.

No deben sustituir enums, estados o cantidades.

27.2. Comparación

Usar comparación ordinal para identificadores técnicos:

```
string.Equals(  
    left.Value,  
    right.Value,  
    StringComparison.Ordinal);
```

La comparación cultural se reserva para texto humano en Presentation.

27.3. Normalización

La normalización debe ocurrir en el boundary propietario:

- trim;
- casing;
- validación de caracteres;
- longitud;
- prefijo.

No normalizar de forma diferente en cada repository o pantalla.

27.4. Localización

El código de dominio devuelve razones y datos, no frases para usuario.

Incorrecto:

```
return "No tienes dinero suficiente";
```

Correcto:

```
return PurchaseResult.Failure(  
    PurchaseFailureReason.InsufficientFunds);
```

Presentation mapea a una clave:

```
purchase.error.insufficient-funds
```

27.5. Interpolación

La interpolación es adecuada fuera de loops calientes para logs o texto técnico. Para UI localizada, utilizar argumentos de localización en lugar de concatenar frases.

27.6. Formato de dinero y números

El dominio conserva valor exacto. Presentation decide:

- símbolo;
- separador decimal;
- miles;
- idioma;
- plural.

Nunca guardar el string formateado como valor económico.

27.7. Rutas

Construir rutas con `Path.Combine` y APIs de plataforma. No concatenar separadores manualmente.

Validar nombres de slot y evitar que una entrada se convierta en path traversal.

28. Threading, concurrencia y acceso a Unity

28.1. Hilo principal

Las APIs de Unity deben considerarse restringidas al hilo principal salvo documentación explícita que indique lo contrario.

No acceder desde un worker a:

- `GameObject`;
- `Transform`;
- `MonoBehaviour`;
- `AssetDatabase`;
- escenas;
- la mayoría de APIs `UnityEngine`.

28.2. Estado compartido

La baseline actual es principalmente single-threaded. No introducir locks o concurrencia por anticipación.

Si se introduce trabajo paralelo:

- definir propietario del estado;
- no compartir entidades mutables sin protección;
- copiar DTOs de entrada;
- retornar datos puros;
- aplicar mutación en el hilo propietario;

- probar cancelación y cierre.

28.3. Save en background

Una futura serialización en background debe separar:

1. captura de IntegratedGameStateSnapshot en estado estable;
2. serialización de datos puros;
3. escritura atómica;
4. notificación al hilo principal.

No capturar GameObject o colecciones mutables mientras se modifican.

28.4. Reentrancia

Un servicio que puede recibir dos comandos antes de completar debe protegerse mediante:

- gate;
- transaction ID;
- estado InProgress;
- cola explícita;
- cancelación.

No confiar solo en deshabilitar un botón visual.

28.5. Transiciones de escena

SceneTransitionGate representa el patrón correcto: una transición en curso debe rechazar o serializar solicitudes concurrentes. La UI debe representar el estado, pero la protección debe vivir en el servicio.

29. Seguridad de datos y robustez de archivos

29.1. Entrada no confiable

Todo archivo persistido debe tratarse como entrada potencialmente inválida:

- JSON malformado;
- campos ausentes;
- números fuera de rango;
- enum desconocido;
- checksum incorrecto;
- generación antigua;
- archivo parcial.

No asumir que un archivo propio siempre es válido.

29.2. Límites

Antes de reservar memoria o crear entidades desde save, limitar:

- longitud de listas;
- tamaño de strings;
- cantidades;
- número de clientes;
- número de objetos colocados;
- profundidad de datos;

- versión de schema.

29.3. Archivos temporales

Los nombres temporales deben evitar colisiones y permanecer en la misma unidad cuando una operación de rename atómico lo requiera.

29.4. Backups

No eliminar un backup válido hasta confirmar el nuevo primario.

29.5. Errores de permisos y disco

Deben mapearse a resultados recuperables cuando sea posible:

- acceso denegado;
- disco lleno;
- directorio ausente;
- archivo bloqueado;
- nombre inválido.

29.6. Datos de usuario

No incluir información personal innecesaria en saves, logs o telemetría. Una futura telemetría requiere política y consentimiento adecuados.

30. Código generado y dependencias externas

30.1. Código generado

No editar manualmente archivos generados cuando el generador vaya a sobrescribirlos.

Deben identificarse mediante:

- comentario de cabecera;
- carpeta clara;
- documentación del comando de regeneración.

30.2. Paquetes externos

Antes de añadir un paquete:

- resolver necesidad concreta;
- revisar licencia;
- comprobar compatibilidad con Unity;
- evaluar mantenimiento;
- medir tamaño y rendimiento;
- definir boundary;
- evitar que tipos externos invadan Domain.

30.3. Wrappers

Una dependencia externa cambiante debe envolverse tras una interfaz propia cuando:

- aparece en varios módulos;
- condiciona tests;

- puede sustituirse;
- su API no coincide con el dominio.

30.4. Copiar código externo

No copiar snippets sin comprender licencia y comportamiento. Toda adaptación debe seguir naming, error handling y tests del proyecto.

31. Deprecación y migración de APIs

31.1. Obsolete

[Obsolete] puede utilizarse para una migración controlada. El mensaje debe indicar la alternativa.

```
[Obsolete(
    "Use StoreInitialSceneContext references instead.")]
public Transform FindCheckoutByName()
{
    ...
}
```

No dejar APIs obsoletas indefinidamente sin fecha o condición de retirada.

31.2. Migración gradual

Secuencia recomendada:

1. introducir contrato nuevo;
2. añadir adapter temporal;
3. migrar consumidores;
4. ejecutar regresión;
5. retirar adapter;
6. actualizar documentación.

31.3. Compatibilidad de save

Una clase o propiedad puede renombrarse en código sin cambiar el campo persistido, o debe incluir migración explícita. No confiar en que el serializer deduzca intención.

31.4. Bridges temporales

Un bridge debe declarar:

- origen legado;
- destino objetivo;
- condición de retirada;
- tests de equivalencia;
- logging de fallback.

Phase1PlacementCompatibilityBridge y fallbacks de Sprint 16 no deben convertirse en arquitectura permanente por omisión.

32. Documentación de decisiones técnicas

32.1. Cuándo crear ADR

Crear o actualizar ADR cuando una decisión:

- cambia el grafo de assemblies;
- introduce tecnología externa;
- cambia schema o estrategia de save;
- altera atomicidad;
- introduce threading;
- cambia composición de escenas;
- establece patrón transversal;
- tiene alternativas con trade-offs relevantes.

32.2. Contenido mínimo

- contexto;
- decisión;
- alternativas;
- consecuencias;
- riesgos;
- estado;
- fecha;
- relación con tests y migración.

32.3. Código y ADR

El código debe reflejar la decisión. Un ADR no justifica una implementación que ya se ha desviado sin registrar la modificación.

33. Patterns aprobados

33.1. Service de aplicación

```
public sealed class InventoryTransferService
{
    private readonly ProductDefinitionRegistry _products;

    public InventoryTransferService(
        ProductDefinitionRegistry products)
    {
        _products = products ??
            throw new ArgumentNullException(nameof(products));
    }

    public InventoryTransferResult Transfer(
        InventoryContainer source,
        InventoryContainer destination,
        ProductDefinitionId productId,
        Quantity quantity)
    {
        // Preflight completo, luego commit.
    }
}
```

33.2. Repository adapter

```
public sealed class JsonIntegratedSaveRepository :
    IIntegratedSaveRepository
{
    private readonly string _rootPath;
    private readonly IntegratedSaveJsonCodec _codec;

    public IntegratedSaveRepositoryResult Write(...)
    {
        // I/O, envelope, backup y promoción atómica.
    }
}
```

No introducir reglas de gameplay dentro del repository.

33.3. View/controller

```
public sealed class CheckoutStationView : MonoBehaviour
{
    [SerializeField]
    private Transform _queueAnchor;

    private CheckoutService _checkoutService;

    public void Configure(
        CheckoutService checkoutService)
    {
        _checkoutService = checkoutService ??
            throw new ArgumentNullException(
                nameof(checkoutService));
    }
}
```

33.4. Proyección de UI

El servicio construye un snapshot de presentación. La pantalla no recorre entidades internas ni recalcula reglas.

33.5. Scene context

```
public sealed class StoreInitialSceneContext : MonoBehaviour
{
    [SerializeField]
    private Transform _entranceAnchor;

    [SerializeField]
    private CheckoutStationView _checkoutStation;

    public void ValidateOrThrow()
    {
        if (_entranceAnchor == null)
        {
            throw new InvalidOperationException(
                "StoreInitial entrance anchor is missing.");
        }
    }
}
```

34. Anti-patrones prohibidos

34.1. God Manager

Un tipo que conoce escenas, dinero, inventario, clientes, UI, audio, save y input debe dividirse.

34.2. Singleton indiscriminado

No convertir cada servicio en Instance global.

Un singleton técnico aprobado debe:

- tener lifetime real global;
- impedir duplicados;
- ser creado por composición;
- exponer contrato limitado;
- ser sustituible en tests cuando proceda.

34.3. Service locator

No resolver servicios por string o diccionario global desde cualquier capa.

34.4. Lógica en UI

Prohibido:

- restar dinero desde un botón;
- quitar stock desde una pantalla;
- decidir elegibilidad del cliente por color del widget;
- guardar el índice del dropdown como producto.

34.5. Saves con referencias Unity

Prohibido serializar GameObject, Transform, MonoBehaviour o ScriptableObject como estado de partida.

34.6. Strings como estado

No usar:

```
if (state == "Closing")
```

Usar enum o tipo.

34.7. Flags incompatibles

No representar una máquina de estados mediante bools independientes.

34.8. Búsquedas globales como DI

No usar FindFirstObjectByType en cada acceso.

34.9. Excepciones ignoradas

No catch vacío ni retorno silencioso ante corrupción.

34.10. Mutación pública de colecciones

No devolver diccionarios o listas internas mutables.

34.11. PlayerPrefs para estado de partida

Solo puede utilizarse para preferencias pequeñas globales cuando se apruebe. No para slot, inventario o economía.

34.12. Código muerto protegido por comentario

Eliminar código muerto una vez que la migración está cerrada. El control de versiones conserva historia.

34.13. Refactor oportunista de gran escala

No reescribir un sistema estable durante un fix pequeño sin necesidad y cobertura.

35. Warnings, analyzers y calidad estática

35.1. Warnings

Una contribución no debe introducir warnings nuevos.

No se desactiva un warning globalmente para ocultar un problema local.

Si se suprime:

- alcance mínimo;
- motivo documentado;
- enlace o issue;
- revisión futura.

35.2. Estado de automatización

No se ha encontrado `.editorconfig` ni analyzer propio en la snapshot revisada.

Plan recomendado, sujeto a sprint aprobado:

1. crear `.editorconfig` en raíz;
2. fijar indentación, newline y naming básico;
3. activar severidades gradualmente;
4. no convertir miles de avisos históricos en bloqueo de una vez;
5. establecer baseline;
6. hacer obligatorio “no warnings nuevos”;
7. evaluar analyzers compatibles con Unity;
8. integrar en CI cuando exista.

35.3. Regla de adopción

El primer cambio de analyzer debe ser un sprint o tarea explícita. No introducirlo dentro de una feature sin estimar impacto.

36. Refactorización y deuda

36.1. Refactor seguro

Un refactor:

- conserva comportamiento;

- tiene tests verdes antes y después;
- evita cambiar schema salvo objetivo explícito;
- evita cambios masivos de GUID;
- mantiene build;
- registra impacto.

36.2. Refactor por seams

Preferir:

- extraer método;
- extraer service;
- introducir interfaz en boundary;
- separar builder de validator;
- separar proyección de vista;
- encapsular colección;
- mover reglas desde MonoBehaviour a Application/Domain.

36.3. Deuda observada

La auditoría identifica como deuda o candidato de revisión:

- ausencia de .editorconfig y analyzers;
- archivos de Editor y runtime por encima de 1.000 líneas;
- búsquedas globales y por jerarquía en integración transitoria;
- dependencia Infrastructure.InputSystem → Presentation;
- coexistencia de composición procedural y autorada;
- composition roots de fase que pueden crecer;
- documentación XML limitada en API pública;
- varios tipos públicos por archivo en agregados grandes;
- builders e installers históricos que deben retirarse o aislarse tras migración.

Ninguna de estas observaciones implica que el sistema esté roto. Indican coste futuro y deben priorizarse según riesgo, no por estética.

36.4. No refactorizar por métrica aislada

Un archivo largo no se divide si la división rompe cohesión o aumenta acoplamiento. Se analiza primero.

37. Revisión de código

37.1. Checklist funcional

- ¿La implementación coincide con GDD y especificación?
- ¿La operación protege invariantes?
- ¿El fallo deja estado intacto?
- ¿Se conserva idempotencia?
- ¿Se actualiza persistencia?
- ¿Existe feedback traducible?

37.2. Checklist arquitectónico

- ¿La capa es correcta?
- ¿Domain permanece libre de Unity?
- ¿Application permanece libre de MonoBehaviour?

- ¿Presentation evita autoridad de negocio?
- ¿La dependencia es explícita?
- ¿Se introdujo un ciclo de assembly?
- ¿La composición está en el root apropiado?

37.3. Checklist Unity

- ¿Las referencias están serializadas o inyectadas?
- ¿Se valida escena/prefab?
- ¿Se preservan .meta?
- ¿Se limpia lifecycle?
- ¿Se desuscriben eventos?
- ¿Hay trabajo innecesario por frame?
- ¿Se depende de nombres?

37.4. Checklist datos

- ¿ID estable?
- ¿schema?
- ¿default?
- ¿migración?
- ¿round trip?
- ¿backup?
- ¿orden determinista?

37.5. Checklist QA

- ¿test nominal?
- ¿límites?
- ¿fallo?
- ¿no mutación?
- ¿regresión?
- ¿PlayMode si afecta escena?
- ¿build si afecta flujo de producción?

37.6. Checklist rendimiento

- ¿se ejecuta por frame?
- ¿asigna?
- ¿busca escena?
- ¿formatea strings?
- ¿instancia material?
- ¿se ha medido?

38. Definition of Done de una contribución C#

Una contribución se considera terminada cuando:

1. compila en los assemblies afectados;
2. no introduce warnings nuevos;
3. respeta dependencias;
4. mantiene una única fuente de verdad;
5. valida argumentos e invariantes;
6. falla sin mutación parcial;
7. utiliza IDs estables;
8. no introduce búsquedas globales estructurales;

9. gestiona lifecycle y suscripciones;
10. no realiza I/O o allocations graves en loops críticos;
11. incluye tests adecuados;
12. mantiene la suite relevante verde;
13. actualiza tests de persistencia si cambia estado;
14. actualiza schema/migración si procede;
15. actualiza escena/prefab sin romper GUID;
16. pasa recorrido manual cuando afecta presentación;
17. pasa build cuando afecta flujo o gate;
18. registra deuda o excepción;
19. actualiza documentación y trazabilidad;
20. puede ser entendida y reproducida sin contexto oral.

39. Política de excepciones

Una excepción al estándar debe registrar:

- regla afectada;
- motivo;
- alternativas consideradas;
- alcance;
- riesgo;
- tests compensatorios;
- condición de retirada;
- owner.

Una excepción no crea un precedente automático.

40. Ejemplos de decisiones comunes

40.1. ¿Result o excepción?

Caso	Decisión
cantidad solicitada mayor que stock	result tipado
constructor recibe repositorio nulo	excepción
archivo de save corrupto	resultado de repository + diagnóstico
referencia obligatoria de escena ausente	excepción/config error antes de gameplay
cliente pierde paciencia	transición normal, no excepción
moneda distinta al sumar	excepción de operación inválida

40.2. ¿Domain o Application?

Código	Capa
Quantity no negativa	Domain
InventoryContainer.TryAdd	Domain
transferir entre dos containers	Application
escribir JSON	Infrastructure
pintar cantidad	Presentation
conectar escena y service	Runtime composition

40.3. ¿ScriptableObject o save?

Dato	Ubicación
definición de producto	ScriptableObject/catalog
stock actual	save/runtime
precio base configurable	catalog/config
precio actual por partida	runtime/save
prefab representativo	catalog asset
cliente activo	runtime/save si el contrato lo exige

40.4. ¿Evento o llamada explícita?

Caso	Mecanismo
UI actualiza saldo después de cambio	evento/proyección
checkout consume reserva y registra dinero	coordinación explícita
audio de venta	evento de presentación
guardado de jornada	service explícito idempotente

41. Plantilla de tipo de dominio

`using System;`

```
namespace VRMGames.CartridgeAndCloud.Domain.Feature
{
    public sealed class FeatureEntity
    {
        private FeatureState _state;

        public FeatureId Id { get; }

        public FeatureState State => _state;

        public FeatureEntity(
            FeatureId id,
            FeatureState initialState)
        {
            if (string.IsNullOrWhiteSpace(id.Value))
            {
                throw new ArgumentException(
                    "Feature ID must be initialized.",
                    nameof(id));
            }

            Id = id;
            _state = initialState;
        }

        public FeatureTransitionResult TryTransition(
            FeatureState target)
        {
            if (!CanTransition(_state, target))
            {
                return FeatureTransitionResult.Failure(
                    FeatureTransitionFailureReason.InvalidState);
            }
        }
    }
}
```

```

    }

    _state = target;
    return FeatureTransitionResult.Success();
}

private static bool CanTransition(
    FeatureState current,
    FeatureState target)
{
    // Tabla de transición explícita.
}
}
}

```

42. Plantilla de componente Unity

```

using System;
using UnityEngine;

namespace VRMGames.CartridgeAndCloud.Presentation.Feature
{
    public sealed class FeatureView : MonoBehaviour
    {
        [SerializeField]
        private Transform _anchor;

        private FeatureService _service;
        private bool _isConfigured;

        public void Configure(FeatureService service)
        {
            if (service == null)
            {
                throw new ArgumentNullException(nameof(service));
            }

            _service = service;
            _isConfigured = true;
        }

        private void Awake()
        {
            if (_anchor == null)
            {
                throw new InvalidOperationException(
                    "Feature anchor is not assigned.");
            }
        }

        private void OnEnable()
        {
            if (_isConfigured)
            {
                _service.StateChanged += OnStateChanged;
            }
        }

        private void OnDisable()
        {
            if (_isConfigured)
            {
                _service.StateChanged -= OnStateChanged;
            }
        }
    }
}

```

```

    private void OnStateChanged(FeatureSnapshot snapshot)
    {
        Render(snapshot);
    }

    private void Render(FeatureSnapshot snapshot)
    {
        // Solo presentación.
    }
}

```

43. Plantilla de test

```

using NUnit.Framework;

namespace VRMGames.CartridgeAndCloud.Tests.EditMode.Feature
{
    public sealed class FeatureServiceTests
    {
        [Test]
        public void Execute_InvalidState_FailsWithoutMutation()
        {
            FeatureEntity entity =
                CreateEntity(FeatureState.Closed);

            FeatureSnapshot before =
                FeatureSnapshot.Capture(entity);

            FeatureResult result =
                CreateService().Execute(entity);

            Assert.That(result.Succeeded, Is.False);
            Assert.That(
                result.FailureReason,
                Is.EqualTo(
                    FeatureFailureReason.InvalidState));
            Assert.That(
                FeatureSnapshot.Capture(entity),
                Is.EqualTo(before));
        }
    }
}

```

44. Plan recomendado para automatizar el estándar

44.1. Fase 1 — Formato base

Introducir `.editorconfig` con:

- UTF-8;
- LF o política acordada;
- cuatro espacios;
- trim trailing whitespace;
- final newline;
- naming básico;
- severidad informativa inicialmente.

44.2. Fase 2 — Warnings nuevos

- baseline de warnings existentes;

- impedir warnings nuevos;
- limpiar por assembly;
- no bloquear Sprint 16/17 por una migración cosmética no planificada.

44.3. Fase 3 — Analyzers

Evaluar compatibilidad con Unity y coste:

- analyzers de .NET compatibles;
- reglas de rendimiento Unity;
- naming;
- nullable por fases;
- prohibición de APIs en Domain.

44.4. Fase 4 — CI

Cuando exista CI estable:

- compile;
- EditMode;
- PlayMode seleccionadas;
- validación asmdef;
- scan de referencias prohibidas;
- build gate cuando corresponda.

45. Criterios de aceptación del estándar

ID	Criterio
CS-STD-001	Todo tipo nuevo usa namespace raíz correcto.
CS-STD-002	Domain no referencia Unity.
CS-STD-003	Application no contiene MonoBehaviour.
CS-STD-004	Presentation no es autoridad de estado de negocio.
CS-STD-005	Toda dependencia estructural es explícita.
CS-STD-006	IDs persistentes son estables y no dependen de nombres de escena.
CS-STD-007	Operaciones compuestas validan antes de mutar.
CS-STD-008	Fallos esperables usan razones tipadas.
CS-STD-009	Dinero no usa float/double.
CS-STD-010	Colecciones internas no se exponen mutables.
CS-STD-011	Suscripciones tienen desuscripción simétrica.
CS-STD-012	No hay búsqueda global por frame.
CS-STD-013	ScriptableObject no almacena progreso autoritativo.
CS-STD-014	Campos persistentes nuevos tienen round trip y migración/default.
CS-STD-015	Guardado escribe de forma atómica.
CS-STD-016	Tests verifican no mutación en fallo crítico.
CS-STD-017	Herramientas de Editor son idempotentes.
CS-STD-018	No se introducen warnings nuevos.
CS-STD-019	No existe catch vacío.
CS-STD-020	Una contribución cumple Definition of Done C#.

Anexo A. Assemblies observados

Assembly	Ruta
VRMGames.CartridgeAndCloud.Editor.ProjectOrganization	_Project/Editor/ProjectOrganization
VRMGames.CartridgeAndCloud.Editor	_Project/Editor/VRMGames.Cartridge
VRMGames.CartridgeAndCloud.Application	_Project/Scripts/Application/VRMGa
VRMGames.CartridgeAndCloud.Domain	_Project/Scripts/Domain/VRMGames.C
VRMGames.CartridgeAndCloud.Infrastructure.InputSystem	_Project/Scripts/Infrastructure/In
VRMGames.CartridgeAndCloud.Infrastructure	_Project/Scripts/Infrastructure/VR
VRMGames.CartridgeAndCloud.Presentation	_Project/Scripts/Presentation/VRMG
VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1	_Project/Scripts/Runtime/VerticalS
VRMGames.CartridgeAndCloud.Tests.EditMode	_Project/Tests/EditMode/VRMGames.C
VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode	_Project/Tests/InputSystem/EditMod
VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode	_Project/Tests/InputSystem/PlayMod
VRMGames.CartridgeAndCloud.Tests.PlayMode	_Project/Tests/PlayMode/VRMGames.C

Anexo B. Archivos grandes observados

La tabla no declara que cada archivo deba dividirse. Sirve para priorizar revisión de cohesión.

Líneas	Archivo
2752	_Project/Editor/ProjectOrganization/RepresentativeAssetIntegrationTool.cs
1949	_Project/Editor/ProjectOrganization/ProjectAssetOrganizationMigration.cs
1454	_Project/Scripts/Infrastructure/UIUX/StoreHudScreen.cs
1314	_Project/Scripts/Application/VerticalSlicePhase1/Phase1VerticalSliceService.cs
1271	_Project/Tests/PlayMode/VerticalSlicePhase1/Sprint16Phase1RuntimePlayModeTests.cs
1134	_Project/Editor/Sprint5/CCS51StoreShellInstaller.cs
1040	_Project/Scripts/Runtime/VerticalSlicePhase1/Phase1StoreBlockoutBuilder.cs
1002	_Project/Scripts/Runtime/VerticalSlicePhase1/RepresentativeStoreVisualBuilder.cs
970	_Project/Scripts/Infrastructure/Persistence/IntegratedSaveJsonCodec.cs
958	_Project/Scripts/Infrastructure/UIUX/MainMenuSlotScreen.cs
934	_Project/Tests/EditMode/VerticalSlicePhase1/Phase1VerticalSliceServiceTests.cs
897	_Project/Editor/Sprint5/CCS53StorePlacementIntegrationInstaller.cs
851	_Project/Scripts/Runtime/VerticalSlicePhase1/Phase1OperationsScreen.cs
701	_Project/Scripts/Presentation/Placement/PlacementRuntimeController.cs
611	_Project/Scripts/Domain/Persistence/IntegratedGameStateSnapshot.cs
608	_Project/Scripts/Runtime/VerticalSlicePhase1/Phase1PlacementCompatibilityBridge.cs
602	_Project/Editor/ProjectOrganization/ProjectAssetOrganizationPlayModeRepair.cs
587	_Project/Editor/Sprint5/CCS54StoreFoundationClosureInstaller.cs
544	_Project/Scripts/Domain/Persistence/SaveRecords.cs
522	_Project/Scripts/Runtime/VerticalSlicePhase1/Sprint16Phase1RuntimeRoot.cs
521	_Project/Tests/EditMode/Persistence/SaveRecoveryTests.cs
514	_Project/Scripts/Application/VerticalSlicePhase1/IntegratedSnapshotPhase1Mutator.cs
504	_Project/Scripts/Runtime/VerticalSlicePhase1/Phase1CharacterLoopController.cs
504	_Project/Scripts/Infrastructure/UIUX/Sprint15RuntimeCompositionRoot.cs
490	_Project/Scripts/Application/UIUX/StoreUiProjectionService.cs

Anexo D. Regla de mantenimiento

Guardar este documento como:

Documentacion/
└─ 09_CSharp_Coding_Standards.md

Debe actualizarse cuando:

- cambie Unity o la versión de C#;
- se adopte .editorconfig o analyzers;
- cambie el grafo de assemblies;
- se apruebe una nueva política de persistencia;
- se incorpore async/Jobs/Burst de forma estructural;
- cambie la estrategia de input;
- se cierre la migración procedural de StoreInitial;
- una retrospectiva detecte una regla repetidamente insuficiente;
- se apruebe una excepción de alcance general.

Estado del documento: fuente vigente de estándares C# para la carpeta Documentacion/.

Anexo E. Matriz de aplicación práctica

Este anexo resume cómo aplicar la norma al trabajo cotidiano. No sustituye las secciones anteriores.

E.1. Cambio de dominio

Cuando se modifica una entidad, value object o policy:

1. confirmar la invariante en GDD, Vertical Slice o Modelo de Datos;
2. mantener Domain libre de Unity;
3. validar construcción y transiciones;
4. usar resultado tipado para rechazo esperado;
5. comprobar igualdad y default de structs;
6. añadir casos nominales, límites y no mutación;
7. revisar impacto de persistencia;
8. ejecutar EditMode completa relevante.

Ejemplo: añadir una nueva razón de abandono no debe resolverse con un string en UI. Debe incorporarse al enum o modelo correspondiente, propagarse mediante Application, proyectarse en Presentation, persistirse si forma parte del historial y probarse.

E.2. Cambio de Application

Cuando se modifica un service o caso de uso:

- listar todas las entidades que muta;
- realizar preflight completo;
- definir orden de commit;
- proteger idempotencia;
- traducir fallos de capas inferiores sin perder causa;
- evitar I/O directo salvo contrato explícito;
- probar estados incompatibles;
- comprobar que el fallo no altera ninguna entidad.

Si el service necesita consultar una escena o un botón, la frontera es incorrecta. Debe recibir un dato, snapshot o contrato.

E.3. Cambio de Presentation o UI

Cuando se modifica un MonoBehaviour, pantalla o controlador:

- confirmar contexto de input;
- comprobar EventSystem cuando exista puntero;
- validar referencias serializadas;
- mantener lógica autoritativa fuera;
- suscribir/desuscribir simétricamente;
- impedir trabajo después de destruir el objeto;
- probar apertura, cierre, foco y error;
- revisar localización y accesibilidad;
- ejecutar PlayMode y recorrido manual.

Una vista puede transformar un Money en texto, pero no decidir el saldo. Puede seleccionar un ID, pero no modificar directamente el inventario.

E.4. Cambio de escena o prefab

Cuando se modifica StoreInitial.unity o un prefab funcional:

1. preservar .meta y GUID;
2. revisar overrides;
3. validar StoreInitialSceneContext;
4. comprobar colliders y navegación;
5. confirmar que no aparece shell duplicado;
6. ejecutar smoke de escena;
7. ejecutar input UI/mundo;
8. guardar/cargar estado representativo;
9. ejecutar Golden Path si afecta anchors;
10. validar build cuando el gate lo exige.

La apariencia no puede corregirse moviendo un anchor lógico sin revisar consumidores y saves.

E.5. Cambio de persistencia

Todo cambio en IntegratedGameStateSnapshot, records, codecs o repositorios debe responder por escrito:

- ¿qué campo cambia?
- ¿qué valor recibe un save anterior?
- ¿cambia schema?
- ¿cómo se valida?
- ¿qué sucede si falta?
- ¿qué sucede si está corrupto?
- ¿el backup sigue siendo válido?
- ¿el orden de listas es determinista?
- ¿la restauración puede fallar antes de mutar?

Pruebas mínimas:

- round trip;
- schema actual;
- schema anterior;
- valor límite;
- corrupción;
- primario inválido y backup válido;
- error de escritura;
- equivalencia observable después de cargar.

E.6. Cambio de herramienta de Editor

Una herramienta debe probarse sobre:

- proyecto limpio;
- estado ya configurado;
- estado parcial;
- referencia ausente;
- asset incompatible.

Debe informar qué cambió y poder reejecutarse sin duplicar. Si la herramienta sustituye una migración manual, debe existir un checklist o test de salida.

E.7. Corrección urgente

Una corrección urgente no queda exenta de la norma. Puede reducir el alcance, pero debe:

- reproducir el fallo;
- clasificar severidad;
- aplicar el cambio mínimo seguro;
- añadir regresión cuando sea viable;
- ejecutar suites afectadas;
- revisar build si es S0/S1 o afecta persistencia;
- registrar deuda si la solución es temporal.

E.8. Revisión de deuda

La deuda técnica se prioriza por riesgo:

1. corrupción o pérdida de datos;
2. bloqueo de Golden Path;
3. dependencia arquitectónica que impide el siguiente sistema;
4. rendimiento medido;
5. coste recurrente de mantenimiento;
6. estilo o legibilidad local.

No debe detenerse Sprint 16 o Sprint 17 para normalizar todo el estilo histórico. Sí debe impedirse que el código nuevo aumente deuda crítica.

E.9. Evidencia de cumplimiento

Una revisión puede demostrar cumplimiento mediante:

- diff pequeño y legible;
- grafo de asmdef sin ciclos;
- Test Runner;
- snapshot antes/después;
- Player.log;
- captura de escena;
- profiler;
- build record;
- ADR;
- checklist de aceptación.

El documento no exige producir todas las evidencias para cada línea de código. Exige seleccionar las que prueban el riesgo real del cambio.